
Diseño Arquitectónico

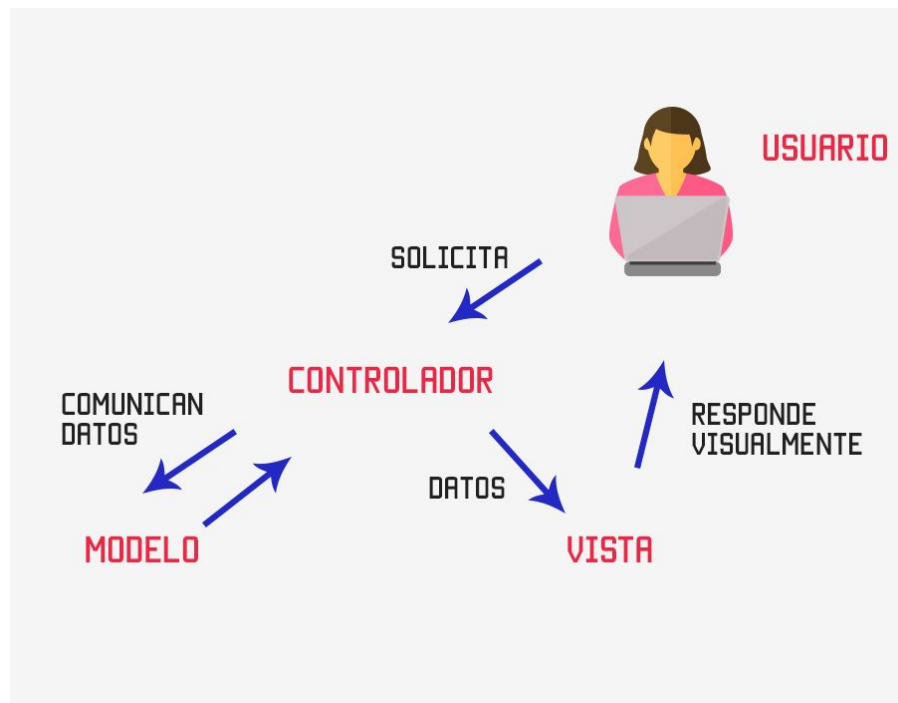
INFO248

Dra. Valeria Henríquez

Modelo-vista-controlador (MVC)

El patrón de diseño MVC es una de las arquitecturas de software más utilizadas en el desarrollo de aplicaciones.

MVC propone la construcción de tres componentes distintos que son el modelo, la vista y el controlador, es decir, por un lado define componentes para la representación de la información, y por otro lado para la interacción del usuario.



MVC

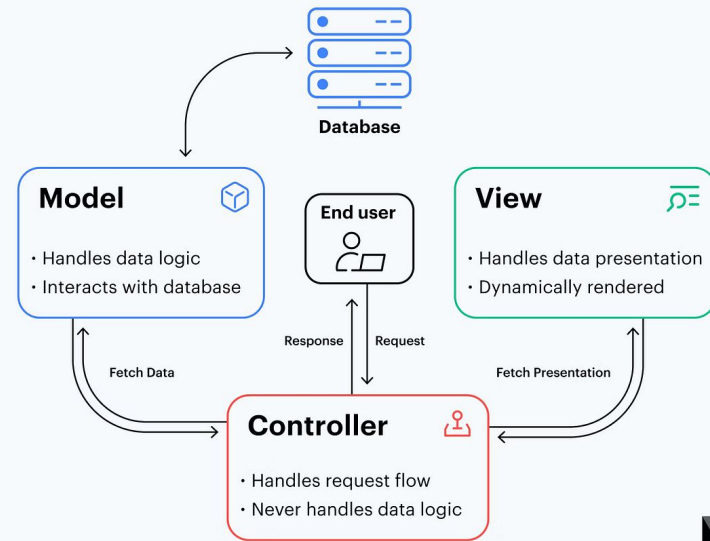
El Modelo: Es la representación de la **información**, por lo tanto gestiona todos los accesos a dicha información, tanto consultas como actualizaciones, implementando también los privilegios de acceso que se hayan descrito en las especificaciones de la aplicación (lógica de negocio).

La Vista: Presenta el 'modelo' (información) en un formato adecuado para **interactuar usualmente con el usuario**.

El Controlador: Responde a **eventos** (usualmente del usuario) e invoca peticiones al 'modelo' cuando se hace alguna solicitud sobre la información. También puede enviar comandos a su 'vista' asociada si se solicita un cambio en la forma en que se presenta el 'modelo'.



MVC Architecture



Resumen - Arquitectura de MVC

Nombre	MVC (modelo de vista del controlador)
Descripción	Separa presentación e interacción de los datos del sistema. El sistema se estructura en tres componentes lógicos que interactúan entre sí. El componente Modelo maneja los datos del sistema y las operaciones asociadas a esos datos. El componente Vista define y gestiona cómo se presentan los datos al usuario. El componente Controlador dirige la interacción del usuario (por ejemplo, teclas oprimidas, clics del mouse, etcétera) y pasa estas interacciones a Vista y Modelo. Véase la figura 6.3.
Ejemplo	La figura 6.4 muestra la arquitectura de un sistema de aplicación basado en la Web, que se organiza con el uso del patrón MVC.
Cuándo se usa	Se usa cuando existen múltiples formas de ver e interactuar con los datos. También se utiliza al desconocerse los requerimientos futuros para la interacción y presentación.
Ventajas	Permite que los datos cambien de manera independiente de su representación y viceversa. Soporta en diferentes formas la presentación de los mismos datos, y los cambios en una representación se muestran en todos ellos.
Desventajas	Puede implicar código adicional y complejidad de código cuando el modelo de datos y las interacciones son simples.

Actividad

Analice la carpeta “MCV python”:

- ¿Qué componentes identifica?
- Cree el diagrama de componentes
- Realice el seguimiento de una operación y cree su diagrama de secuencia.
- Imagine una nueva funcionalidad para el sistema.
- Cree un diagrama de secuencia para implementar dicha funcionalidad
- Implemente la funcionalidad siguiendo el patrón arquitectónico propuesto

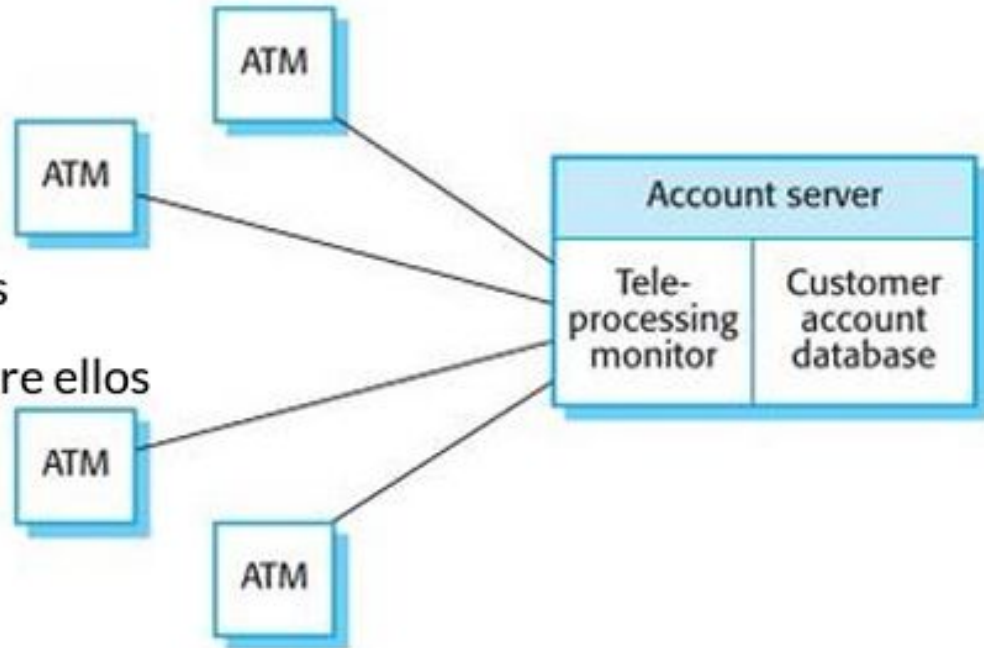
Cliente Servidor

La arquitectura cliente-servidor es un modelo donde un cliente (como un navegador o app) solicita recursos a un servidor, que procesa y responde. Muchas tecnologías web modernas la usan para distribuir carga y centralizar datos

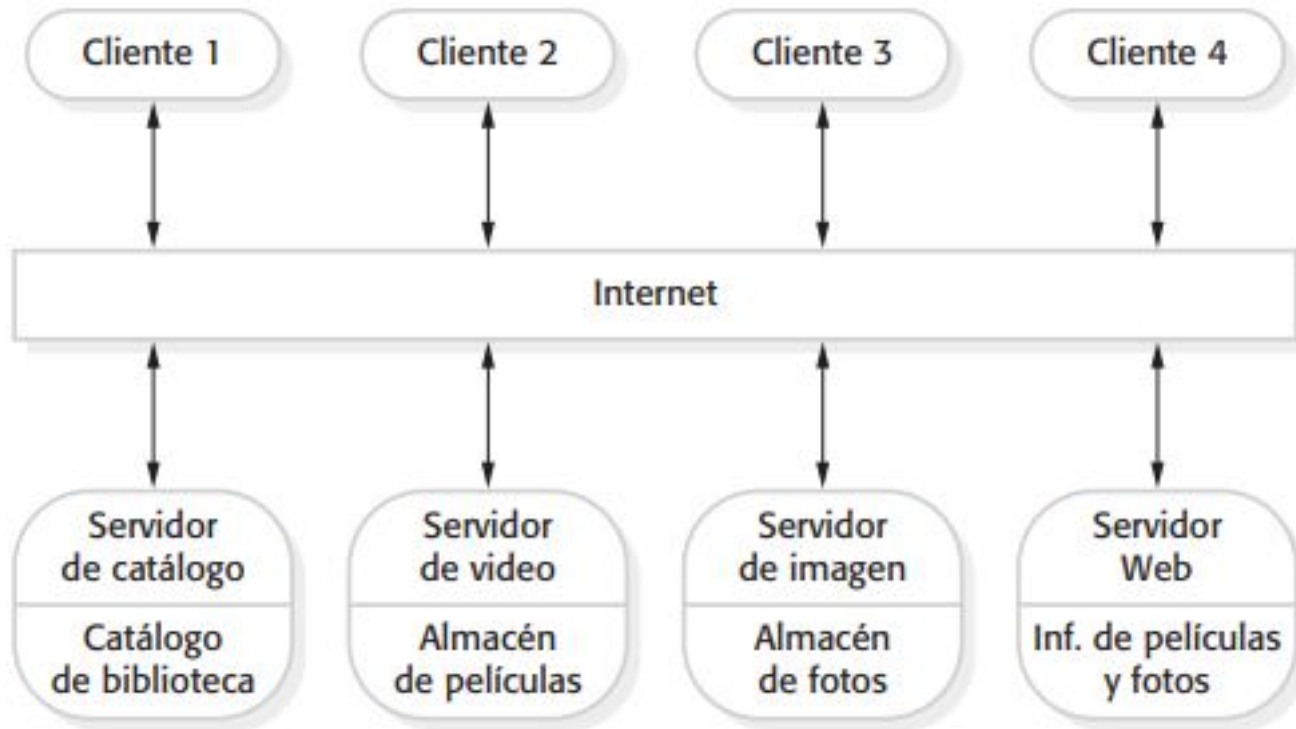
Tecnología	Cliente Ejemplo	Servidor Ejemplo
Web	Navegador Chrome	Apache/Nginx
Correo	Outlook	Servidor SMTP
Bases Datos	PgAdmin	Postgresql

Cliente Servidor

- Múltiples clientes independientes accediendo al mismo server
- Especialización de máquina virtual de 2 capas:
 - Capa 1: Cliente
 - Capa 2: Servidor
 - Conexión por red
- Clientes conocen al Servidor
- El Servidor no conoce a los Clientes
- Los Clientes son independiente entre ellos



Cliente Servidor - Ejemplo



Resumen - Cliente Servidor

Nombre	Cliente-servidor
Descripción	En una arquitectura cliente-servidor, la funcionalidad del sistema se organiza en servicios, y cada servicio lo entrega un servidor independiente. Los clientes son usuarios de dichos servicios y para utilizarlos ingresan a los servidores.
Ejemplo	La figura 6.11 es un ejemplo de una filmoteca y videoteca (videos/DVD) organizada como un sistema cliente-servidor.
Cuándo se usa	Se usa cuando, desde varias ubicaciones, se tiene que ingresar a los datos en una base de datos compartida. Como los servidores se pueden replicar, también se usan cuando la carga de un sistema es variable.
Ventajas	La principal ventaja de este modelo es que los servidores se pueden distribuir a través de una red. La funcionalidad general (por ejemplo, un servicio de impresión) estaría disponible a todos los clientes, así que no necesita implementarse en todos los servicios.
Desventajas	Cada servicio es un solo punto de falla, de modo que es susceptible a ataques de rechazo de servicio o a fallas del servidor. El rendimiento resultará impredecible porque depende de la red, así como del sistema. Quizás haya problemas administrativos cuando los servidores sean propiedad de diferentes organizaciones.

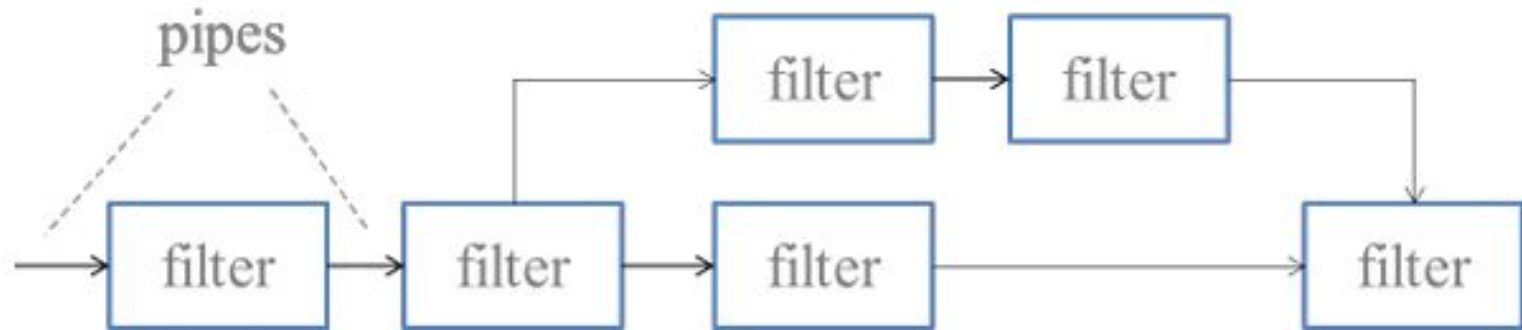
Actividad

Analice la carpeta “cliente-servidor”:

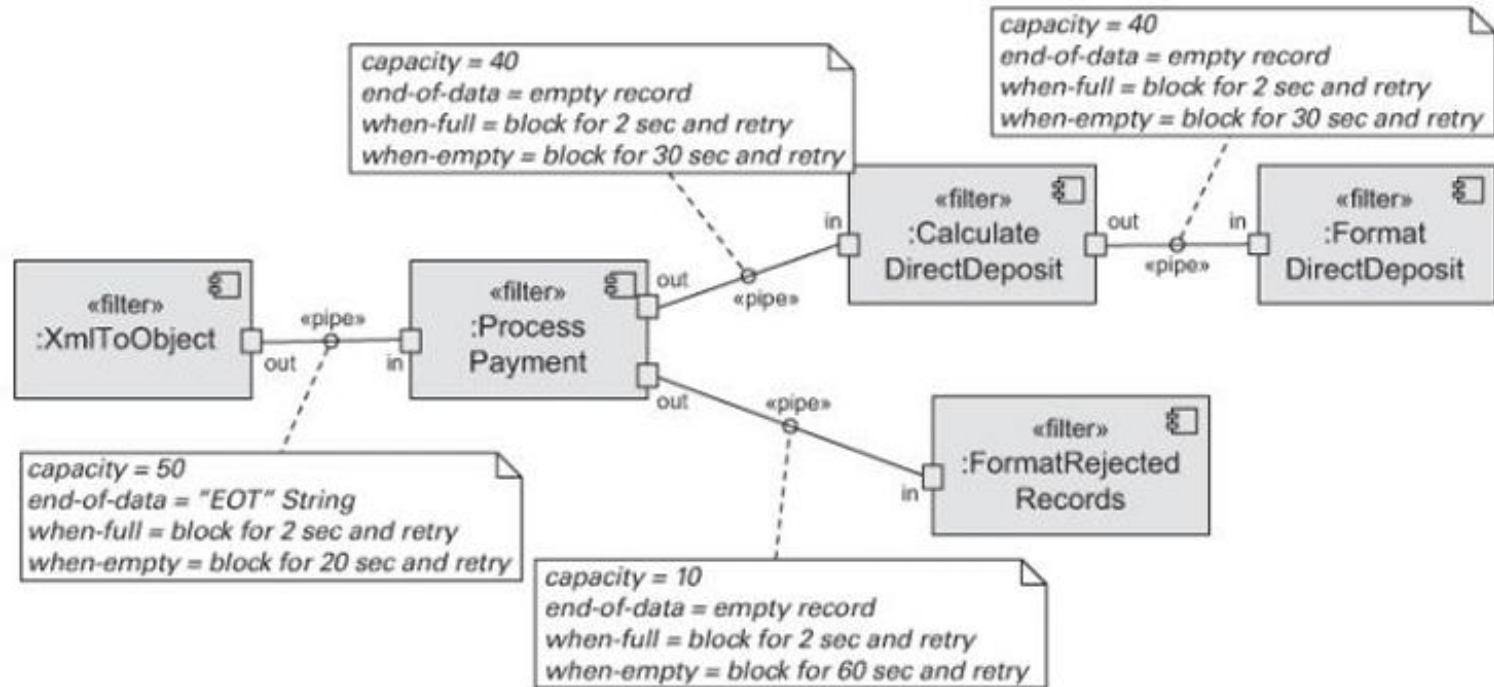
- ¿Qué elementos identifica?
- Realice el seguimiento de una operación y cree el diagrama de secuencia.
- Imagine una nueva funcionalidad para el sistema.
- Cree el diagrama de secuencia para implementar dicha funcionalidad
- Implemente la funcionalidad siguiendo el patrón arquitectónico propuesto

Pipes and Filters (Arquitectura de tubería y filtro)

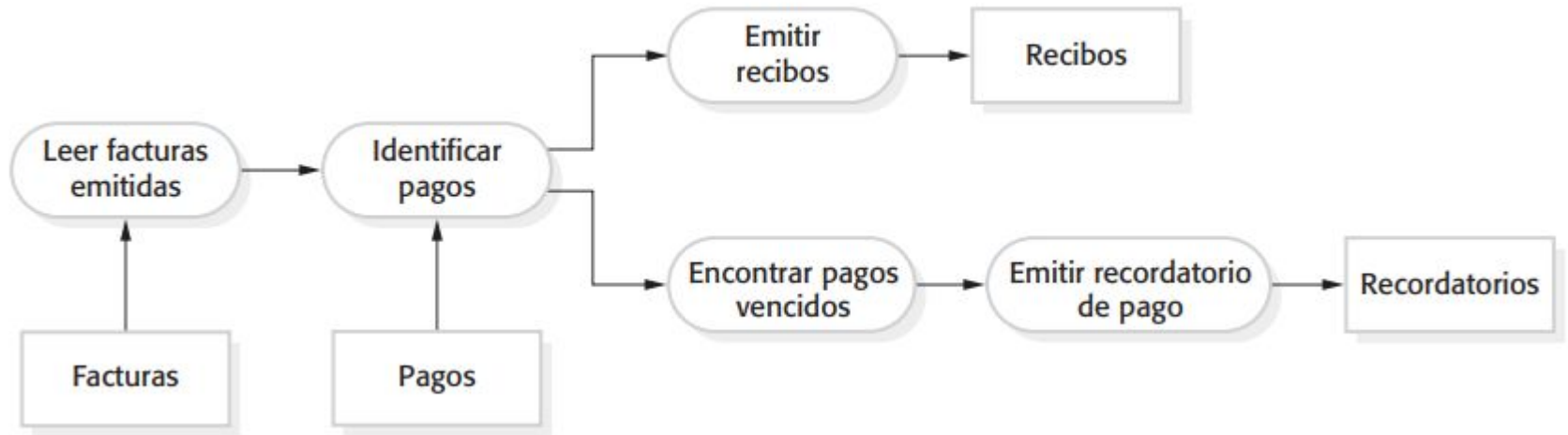
Modelo en tiempo de operación de un sistema, donde las transformaciones funcionales procesan sus entradas y producen salidas. Los datos fluyen de uno a otro y se transforman conforme se desplazan a través de la secuencia. Cada paso de procesamiento se implementa como un transformador. Los datos de entrada fluyen por medio de dichos transformadores hasta que se convierten en salida. Las transformaciones pueden ejecutarse secuencialmente o en forma paralela. Es posible que los datos se procesen por cada transformador ítem por ítem o en un solo lote. Utilizado desde el inicio del uso de computadoras para el procesamiento automático de datos.



Pipes and Filters - Ejemplo



Pipes and Filters - Ejemplo



Otros ejemplos

Ejemplo	Filtros Ejemplo	Pipes
Compilador	Lexer → Parser → Optimizer slideshare	Flujo de tokens AST
ETL Datos	Extract → Transform → Load elemarjr	Streams serializados
TensorFlow Pipeline	Tokenize → Embed → Model → Decode	Streams de tensores

Resumen - Pipes and Filters

Nombre	Tubería y filtro (<i>pipe and filter</i>)
Descripción	El procesamiento de datos en un sistema se organiza de forma que cada componente de procesamiento (filtro) sea discreto y realice un tipo de transformación de datos. Los datos fluyen (como en una tubería) de un componente a otro para su procesamiento.
Ejemplo	La figura 6.13 es un ejemplo de un sistema de tubería y filtro usado para el procesamiento de facturas.
Cuándo se usa	Se suele utilizar en aplicaciones de procesamiento de datos (tanto basadas en lotes [<i>batch</i>] como en transacciones), donde las entradas se procesan en etapas separadas para generar salidas relacionadas.
Ventajas	Fácil de entender y soporta reutilización de transformación. El estilo del flujo de trabajo coincide con la estructura de muchos procesos empresariales. La evolución al agregar transformaciones es directa. Puede implementarse como un sistema secuencial o como uno concurrente.
Desventajas	El formato para la transferencia de datos debe acordarse entre las transformaciones que se comunican. Cada transformación debe analizar sus entradas y sintetizar sus salidas al formato acordado. Esto aumenta la carga del sistema, y puede significar que sea imposible reutilizar transformaciones funcionales que usen estructuras de datos incompatibles.

Actividad

Analice la carpeta “pipe&filter”:

- ¿Qué elementos identifica?
- Realice el seguimiento de una operación y cree el diagrama de secuencia.
- Imagine una nueva funcionalidad para cada sistema.
- Cree el diagrama de secuencia para implementar dicha funcionalidad
- Implemente la funcionalidad siguiendo el patrón arquitectónico propuesto